Technical Datasheet 4G Gateway with 4DI, 4DO, & 4AI

Product Overview: -

The 4G Gateway is an advanced networking device designed to provide seamless connectivity and data management capabilities for various applications. With support for 4G LTE connectivity, cloud connectivity via MQTT protocol, and robust storage options, it offers a comprehensive solution for remote monitoring and control systems. Equipped with an integrated I/O interface featuring **4 Digital Inputs (DI)**, **4 Digital Outputs (DO)**, and **4 Analog Inputs (AI)**, alongside a Modbus RS485 port and micro SIM compatibility, it caters to a wide range of industrial and IoT applications. The intuitive web-based UI interface allows for easy configuration and management, making it an ideal choice for both professionals and enthusiasts.

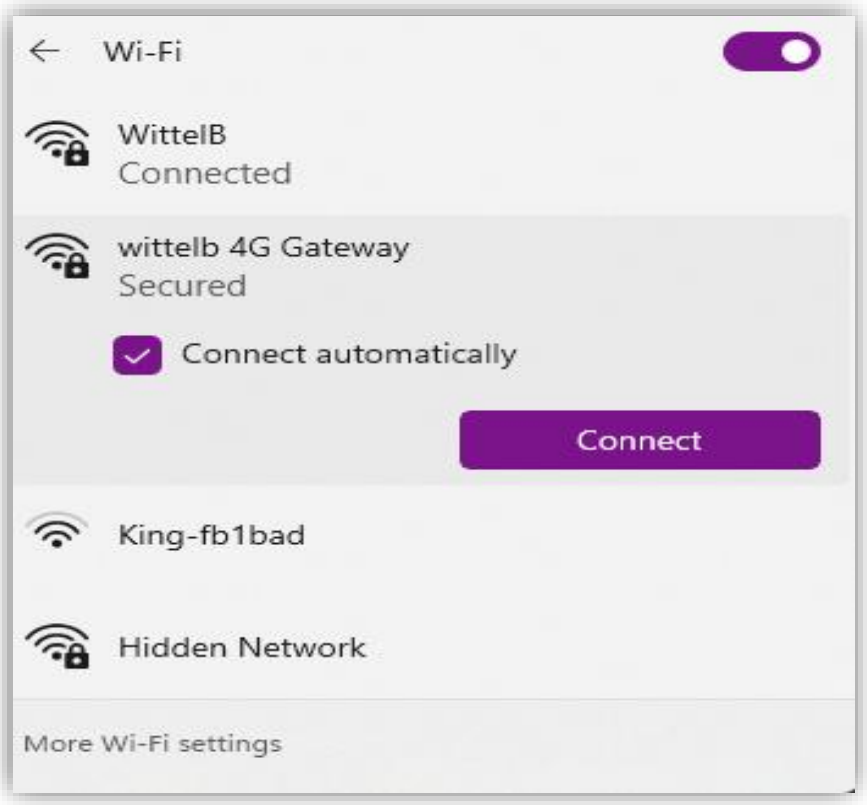


Specifications

General:-

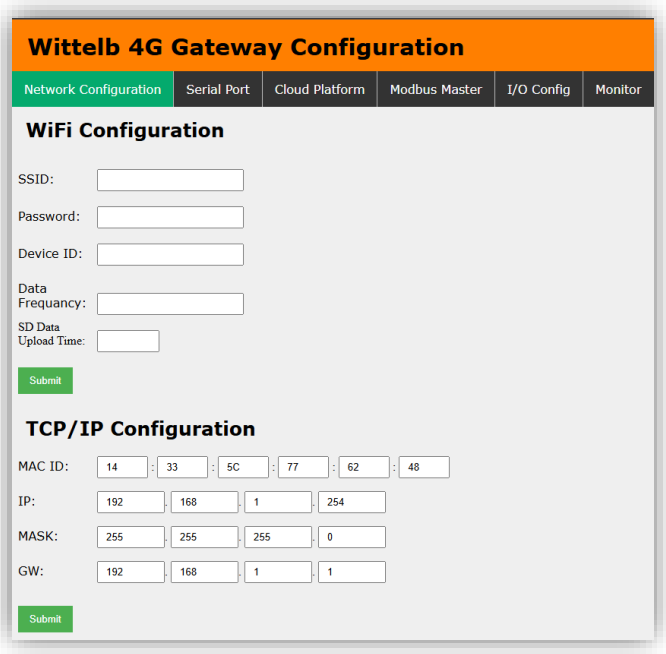
- **4G LTE Connectivity:** Utilizes 4G LTE technology for reliable and high-speed internet access. If 4G is not available, the system automatically switches to Wi-Fi. If both 4G and Wi-Fi are unavailable, the data is securely stored in the SD card for later upload.
- **Cloud Connectivity (MQTT**): Seamlessly connects to cloud platforms using the MQTT protocol for efficient data transmission and management**.**
- **Event Storage:** Offers on board event storage, capable of storing up to 50,000 events or more. Includes an external memory card slot for easy data retrieval in the event of 4G network issues.
- **Modbus RS485 Port:** Features 1 port of Modbus RS485 for integration with industrial control systems and sensors.
- **GPS Location Data**: Provides GPS location data for monitoring of remote equipment.
- **Micro Sim Compatibility:** Supports micro-Sim card for flexible and convenient connectivity option.
- **Web-based UI Interface**: Intuitive web-based user interface for easy configuration, monitoring, and management.
- **Dimensions:** 110 mm L x 110 mm B x 50 mm H.
- **Power:** Input Power – 15-40 VDC, Typical – 12V DC @ 150mA.
- **Operating Temperature:** 0 – 60C (32 ~ 140F).
- **Storage Temperature:** 20 - 70C (-4 ~ 158F).
- **Storage Humidity:** 5 ~ 95 % RH, non – Condensing.

Configuration Settings: -



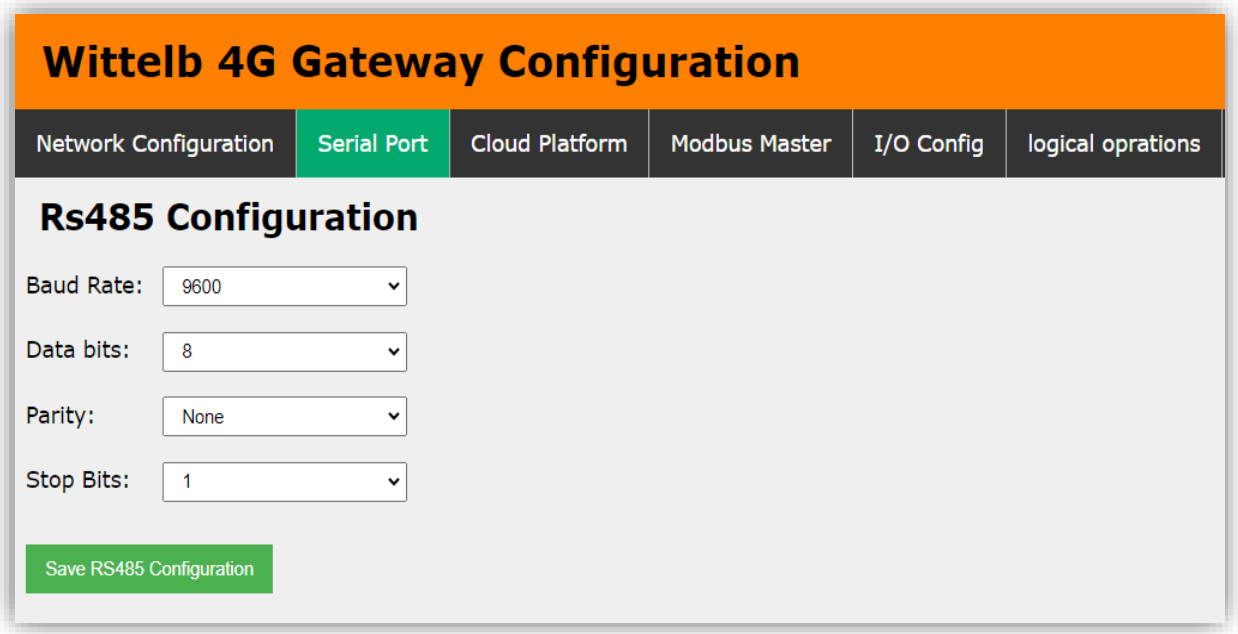
- Power up the module by connecting a 12-24 V DC supply.
- Insert the SIM card into the designated slot.
- Place the Micro SD card into its designated slot.
- Connect MODBUS RTU Multiple Slave devices (up to 31) to the A & B Terminal Of Module.
- Access the module's configuration hotspot (ID: Wittelb-4G-GATEWAY, PASS: 123456789).
- Default IP Address of Module is “192.168.1.100/cpp1”.
- Upon entering the provided IP address, a configuration webpage Pages will be accessible.

Page1:- TCP/IP Configuration:-



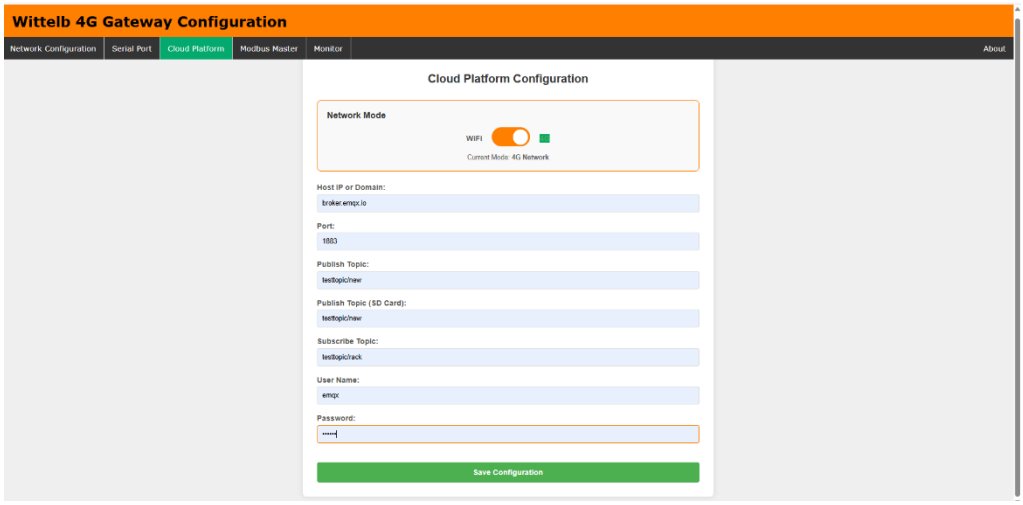
- Enter MAC ID of the Ethernet interface (6 hex bytes, e.g. 14:33:5C:77:62:48).
- Enter IP address for the Ethernet port (default: 192.168.1.254).
- Enter Subnet Mask (default: 255.255.255.0).
- Enter Gateway address (default: 192.168.1.1).
- Click Submit to save settings; the module applies to the new Ethernet configuration immediately.

Page2:- On second page you can set device Baud Rate (9600 / 19200 / 38400 / 57600 / 115200), Data Bits (5 / 6 / 7 / 8), Parity (None / Odd / Even), and Stop Bits (1 / 2) for RS485 Communication



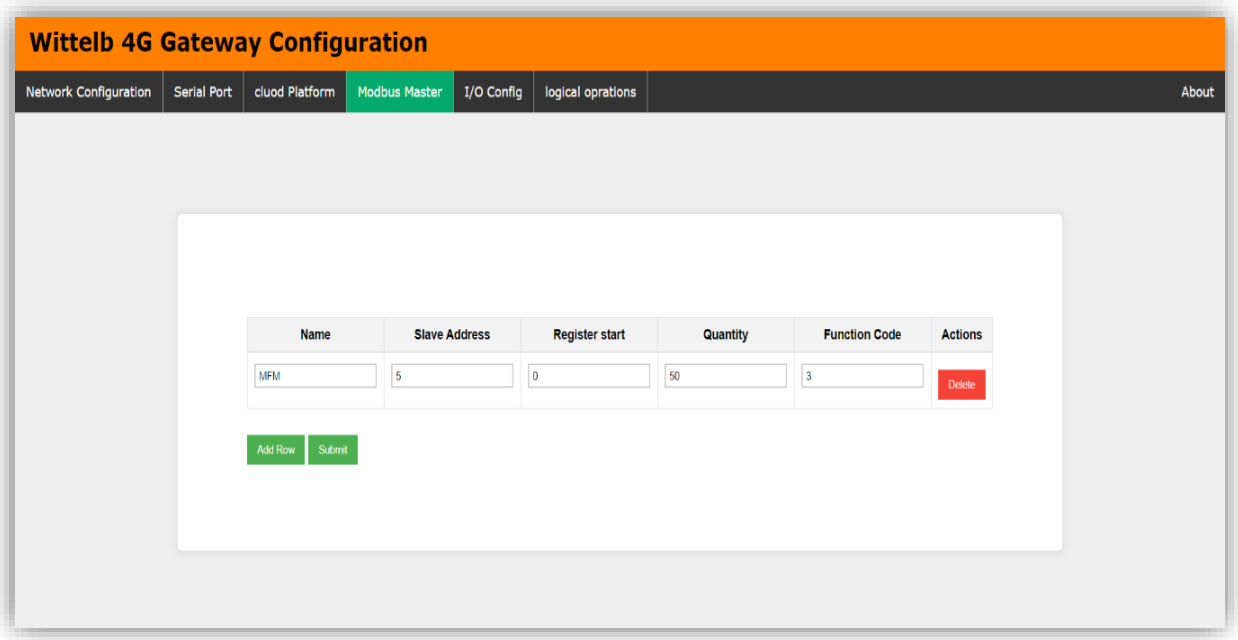
Page3:-

- Navigate to page 3 on the cloud platform.
- Input the Host IP or Domain.



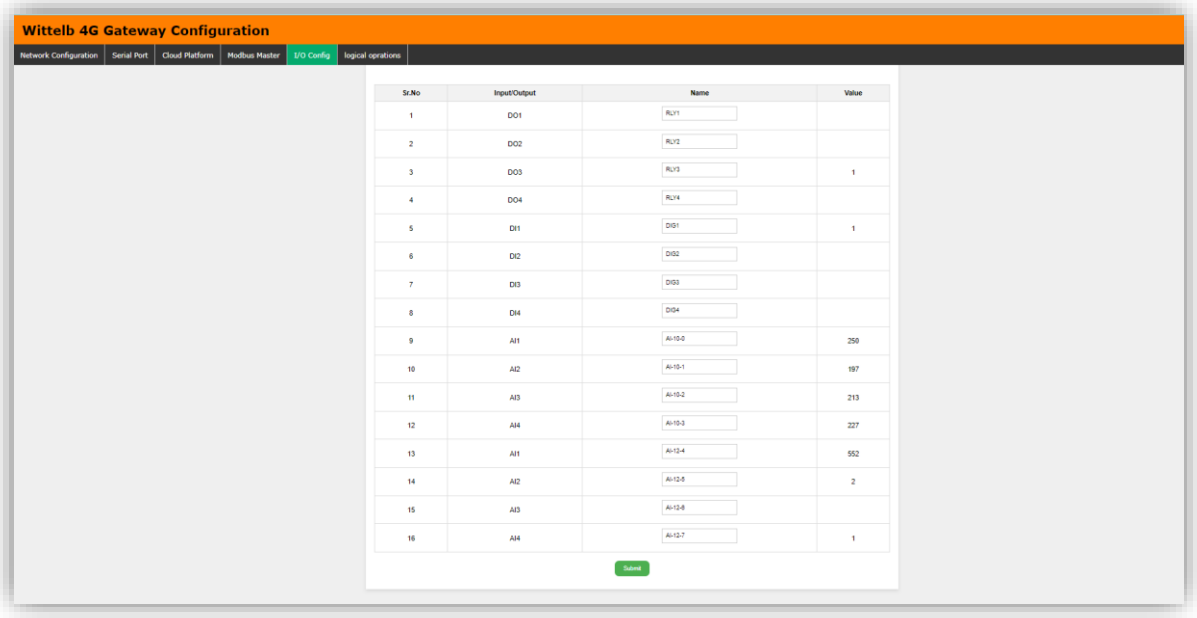
- Enter the port Number.
- Enter the topics for publishing and receiving messages.
- Provide a username and password if required.
- Allow a few seconds for the module to restart and establish a connection to MQTT.

Page4:-



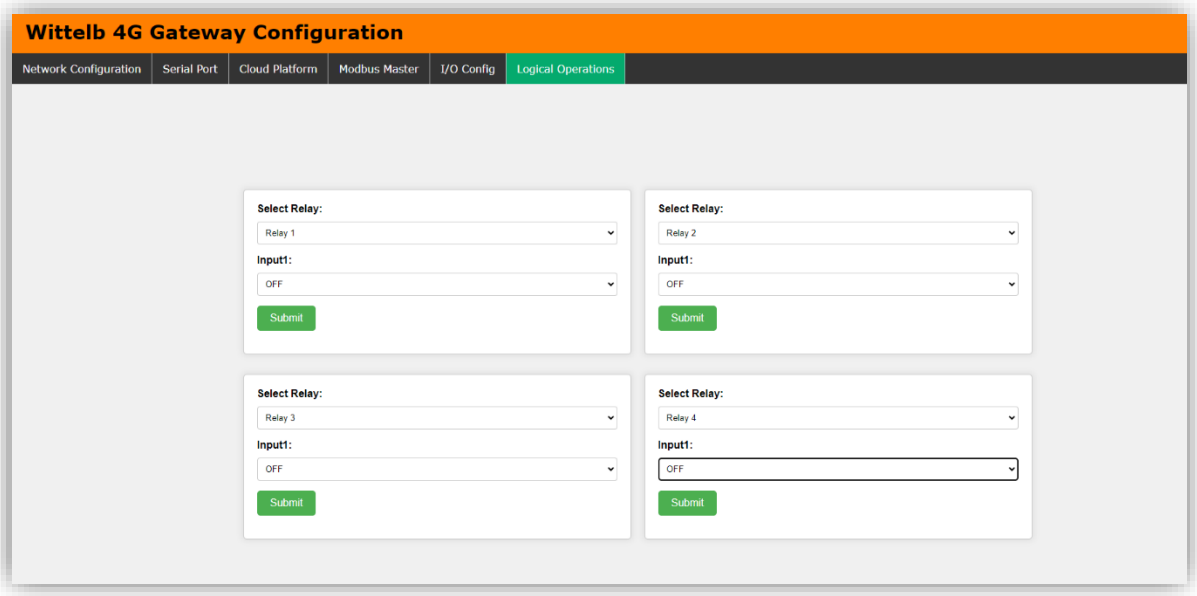
- Go to page 4 on the Modbus Master interface.
- Input the name and slave address for the designated slave.
- Specify the register address from which you want to read slave data.
- Indicate the register end address for the slave.
- Enter the function code corresponding to the desired slave.

Page5:-

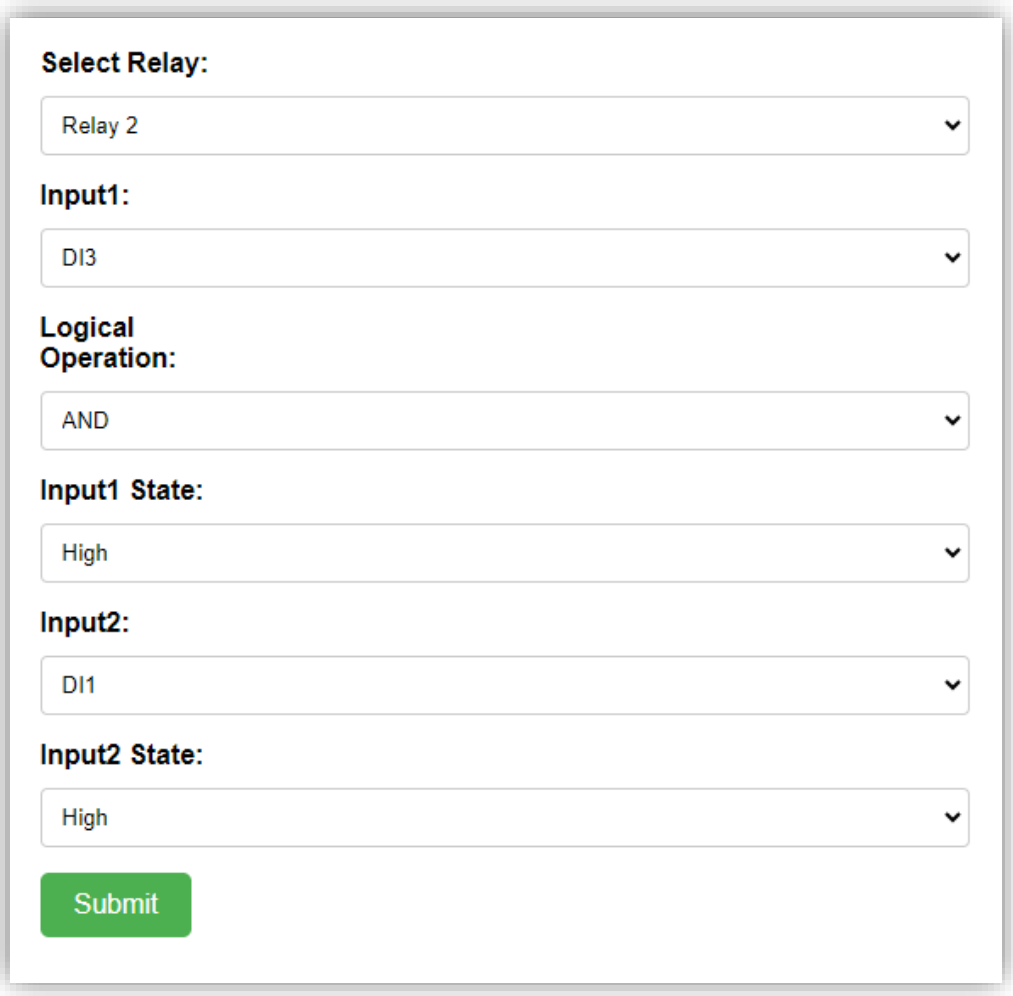


- Go to page 5 on the I/O Config interface.
- You can view real-time values of both analog and digital inputs on the baseboard.
- Additionally, you have the option to assign specific names to each input/output (I/O) as needed.

Page6:-



- Navigate to page 6 on the Logical Operations interface.



- Here, you can configure logical AND and OR operations for each of the 4 relays.
- You have the flexibility to assign any two digital inputs to a relay and choose the desired logical operation.
- Additionally, you can assign an analog input to a relay and define minimum and maximum threshold values for relay operation.
- If you prefer not to use any logical operation, simply select the "OFF" option for input one.

Input & Output module Configuration Settings: -

- The Gateway has 4 Digital Outputs (DO) with Relay of 10 Amps 230V,
- 4 Analog Inputs (AI) to connect with 4-20ma or 0-10 v sensors,

- 4 Digital Inputs (DI) will help monitor the status with logic voltage input between 4-40V DC. Additionally, we have also provided 12 V voltage ref and GND pins for DI detection logic.
- Digital outputs are normally open.
- Data of Digital Inputs (DI) and Analog Inputs (AI) would be directly collected by the Processor and transmitted over 4G to the Cloud based on the Poling frequency.
- Configure each Analog Input (AI) to operate with either 5V or 10V and choose between 10-bit and 12-bit resolution by adjusting the jumper settings.
- You can trigger DO from cloud by sending Following Message

Relay No	ON	OFF
DO1	**"DO1":1**	**"DO1":1**
DO2	**"DO2":1**	**"DO1":1**
DO3	**"DO3":1**	**"DO1":1**
DO4	**"DO4":1**	**"DO1":1**

SMS Command Reference Guide:-

- The device supports remote configuration, register management, and system control via SMS. Below is the complete command syntax, parameter definitions, and expected device behavior.

1. Modbus Slave Register Management

Configure or modify Modbus polling entries using the ADD and UPD commands, and remove them using DEL.

- **Add Syntax:** ADD,idx,name,slaveid,reg,funcode,datatype
- **Update Syntax:** UPD,idx,name,slaveid,reg,funcode,datatype
- **Example:** ADD,0,Temp1,1,100,3,0

Field	Meaning
Idx	Row index (starts at 0)
name	Slave/register name (avoid commas)
slaveid	Modbus slave address (1–247)
reg	Register address (0–65535)
funcode	1 = coil, 3 = holding reg, 4 = input reg
datatype	0 = int16, 1-4 = float formats, 5 = uint16, 6 = int16÷10

Slave Register Delete

- **Syntax:** DEL,idx
- **Example:** DEL,3

2. System Settings & Control

- **Polling Frequency Set:** FREQ,minutes (Example: FREQ,5)
- **RS485 Settings:** RS485,baud,databits,parity,stopbits (Example: RS485,9600,8,None,1)
- **MQTT Broker Change:** BROKER,broker_address (Example: BROKER,mqtt.example.com)
- **Device Reboot:** Send either REBOOT or RESTART to remotely power-cycle the device

3. Proposed Future Features (Pending Implementation)

Arithmetic Logic (Add/Sub/Mul/Div)

- **Syntax:** LOGIC,idx,slaveid,op,reg,offset
- **Example:** LOGIC,0,1,a,100,5.5
- **Delete Command:** LOGICDEL,idx

Field	Meaning
Idx	Logic row index (0–30)
slaveid	Slave address (1–247)
op	a = add, s = subtract, m = multiply, d = divide
reg	Register address
offset	Value used in the operation

Digital Output (DO) Control (Pending Base Board Command Format)

- **Syntax:** DO,channel,value
- **Examples:** DO,1,1 (Turns DO1 ON), DO,3,0 (Turns DO3 OFF)

Field	Meaning
channel	1, 2, 3, or 4 (Targeting DO1, DO2, DO3, or DO4)
value	1 = ON, 0 = OFF

4. Device Reply Behavior

- **Valid Command:** The device replies with an "OK: ..." message.
- **Invalid Format:** The device replies with an "ERR: ..." message.
- **Message Routing:** The SMS reply is always sent directly back to the phone number that originally sent the command.

Modbus Slave Configuration:

The gateway supports reading up to 100 individual Modbus points (e.g., sensor values, coil states, pump statuses), stored as entries indexed from 0 to 99. Each entry requires five specific parameters to successfully poll the connected RS485 field devices.

Configuration Parameters:

- **Slave Index (0-99):** The internal row position for storage on the gateway, which is not transmitted over the Modbus line.
- **Slave Name:** A custom alphanumeric label (up to 19 characters) used as the JSON key during MQTT transmission. If left blank, the device auto-generates a name based on the address (e.g., S1_REG40001).
- **Slave ID:** The hardware RS485 address of the target device, which must be configured between 1 and 31. Multiple entries can share the same Slave ID.
- **Function Code:** Defines the Modbus reading method. The gateway currently supports Code 1 (Coil Read for ON/OFF status), Code 3 (Holding Register Read for settings/data), and Code 4 (Input Register Read for measured values).
- **Data Type:** Defines how the raw register data is parsed (applicable only to Function Codes 3 and 4).
    - **Type 0:** Simple Integer (Reads 1 Register).
    - **Type 1-4:** Float / Decimal Number (Reads 2 Registers). The 1 through 4 selection alters the byte order; if a float value reads incorrectly, rotating through orders 1-4 typically resolves it. Data type is irrelevant for Coil reads, which always yield 0 or 1.

Register Addressing & Offset Logic:

Modbus datasheet addresses (e.g., 40001) are not transmitted directly to the slave device. The first digit indicates the register type (0 = Coil, 3 = Input, 4 = Holding) and is discarded because the Function Code dictates the type.

Standard Modbus protocol requires subtracting 1 from the remaining digits. Because some manufacturers pre-subtract this offset in their documentation, adding or subtracting 1 is the primary troubleshooting step if a register fails to read correctly.

Datasheet Register	Type Prefix Removed	Final Entered Value (Subtract 1)
40001 (Holding)	0001	0
40010 (Holding)	0010	9
30005 (Input)	0005	4
00001 (Coil)	0001	0

MQTT Communication & JSON Payloads:

he gateways acts as an MQTT client, grouping data per Modbus slave address into a single JSON object and transmitting it over the network.

Telemetry Publishing (Device -> Cloud):

- **Primary Topic:** Data is published to publish_topic, defined in the web-config panel.
- **Payload Structure:** Each JSON payload includes the Device ID, a Unix timestamp (sent as a string), the integer Slave Address, and the grouped register values using the configured Slave Names.
- **Data Formatting:** Coil reads produce 0/1 integers, while 16-bit register reads yield plain integers. 32-bit float reads are automatically rounded to 4 decimal places. If a float registers as NaN or infinite, it is transmitted as a null value rather than a number.
- **SD-Card Fallback:** If the MQTT connection drops, the JSON payload is saved to the local SD card. Once connectivity is restored, the buffered data is republished unchanged to a secondary topic (pub_topic1).

Command Reception (Cloud -> Device):

- **Target Topic:** The gateway listens for remote write commands on subscriber_topic.
- **Payload Requirements:** To trigger a write action, the incoming JSON must contain the target slave Address, the specific register, and the value to be written.
- **Execution Limitation:** Currently, the MQTT command engine strictly enforces Modbus Function Code 6 (Write Single Register) for all incoming writes. While the internal firmware supports Coils (FC5) and multi-registers (FC15/FC16), these are not yet mapped to the MQTT listener.

The following table outlines the JSON payloads handled by the MQTT engine, their communication direction, and their associated topics.

	JSON Payload Type	Direction	Topic
1	IO Board Data	Device -> Cloud	publish_topic
2	Modbus Slave Data	Device -> Cloud	publish _topic
3	Write Acknowledgment	Device -> Cloud	publish _topic
4	SD-Card Fallback Data	Device -> Cloud	publish _topic1
5	Register Write Command	Cloud -> Device	subscriber_topic

1. Modbus Slave Data (Device -> Cloud)

Sent once per configured Modbus slave address. This payload groups all registers belonging to that specific slave into a single JSON object. Field keys are derived from the user-configured "Slave Name" (e.g., "Voltage", "Current").


```
{
 "DeviceId": "DEV001",
 "time": "1725600000",
 "slaveAddress": 1,
 "Voltage": 230.5000,
 "Current": 12.3400,
 "PumpStatus": 1
}
```


(Note: "time" is a Unix timestamp formatted as a string. 32-bit float values are rounded to 4 decimal places before transmission.)

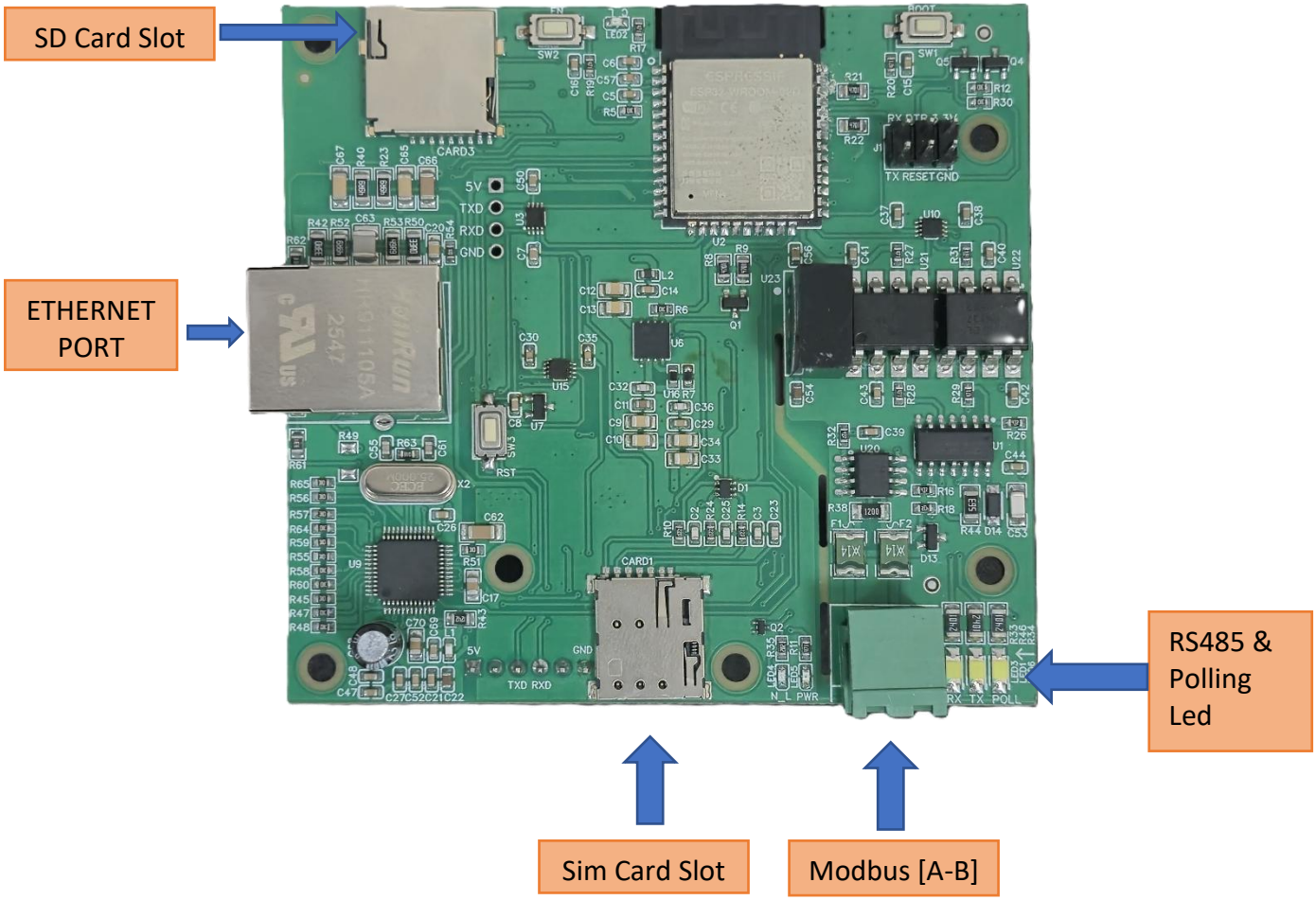
2. Register Write Command (Cloud -> Device)

Sent by the cloud platform to remotely write a value into a holding register. The device will only execute the command if the incoming JSON contains both the register and value keys.


```
{
 "slaveAddress": 1,
 "register": 40010,
 "value": 1
}
```


(Note: Currently, the device processes this incoming JSON by issuing a Modbus Function Code 6 (Write Single Register) regardless of the target register's actual data type.)

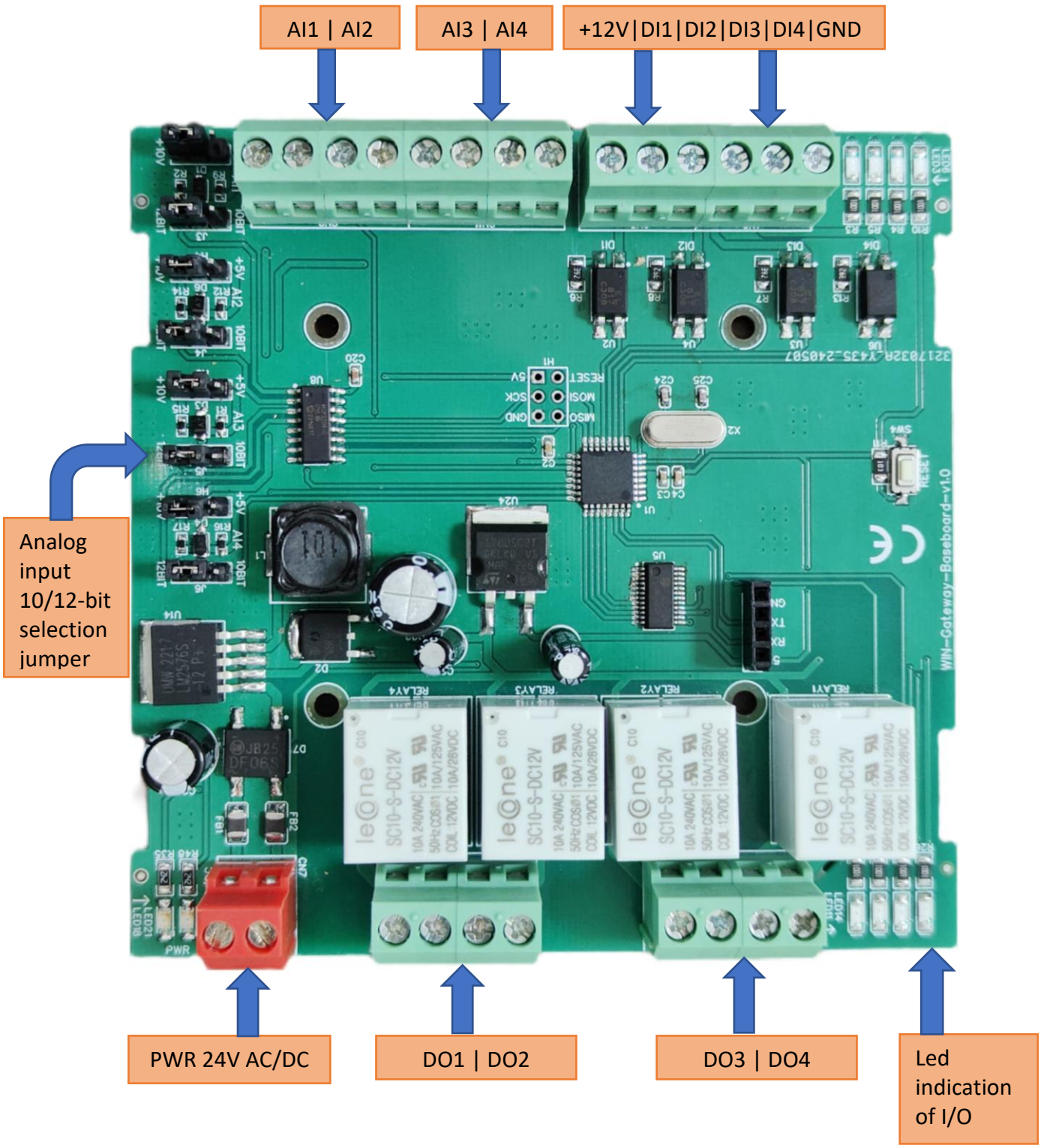
4G Gateway connection Diagram:



Additional Features: -

Communication ports are isolated
Input power reverse polarity safety
ESD Safety IEC 61000-4-2, ± 30KV contact, ± 30KV air
EFT IEC 61000-4-4, 50A (5/50ms)
400V isolation.

WIN-GW-4G-4DDA-M-E



Contact us: -

Augmatic Technologies Pvt. Ltd.,
Plot no 6, Shah Industrial Estate II,
Kotambi,
Vadodara – 391510.
Email – Sales@wittelb.com